PORTAL-NX · BATTLEPLAN

Portal Frontend Rebuild

A strictly layered Nx monorepo: DDD, onion architecture, Redux Toolkit and a frontend that holds no tokens.

Thomas Kortyka
Lead Consultant (CGI)

Hackathon kickoff · November 2026

GOALS

What we are building

Dumb frontend

No Keycloak library, no tokens, no role logic. Login, refresh and permissions live in the new API.

DDD + onion

Frontend only: anemic entities and classic use cases. Dependencies only point inward.

Redux Toolkit

Still the only state tool, but as its own layer between use cases and React.

Everything in libs

Apps are only the composition root: store, DI container, router, providers.

STARTING POINT

Three legacy apps we replace

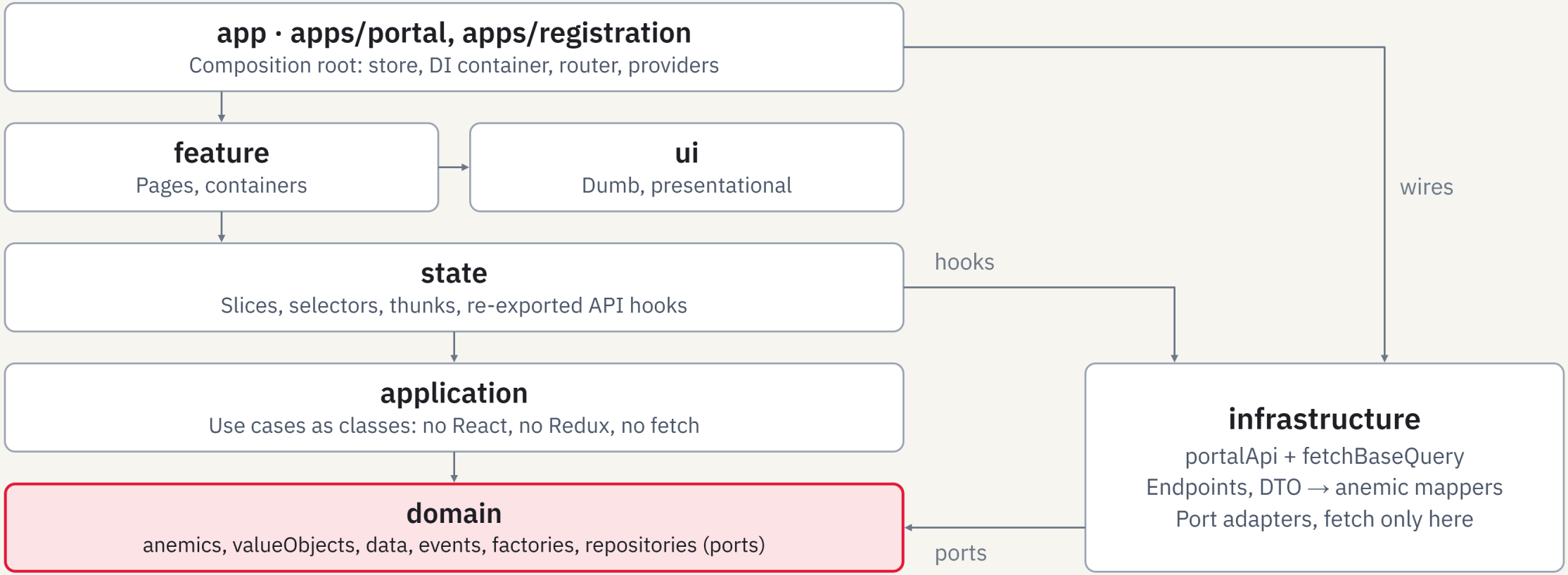
Project	Size	Stack	What stands out
portal-frontend	513 files, ~72,700 lines	React 18, MUI 5, RTK 2.2, Router 6	36+ createApi instances, tokens to 5 backends
portal-frontend-registration	80 files, ~7,300 lines	Router 5, axios, Bootstrap	Second auth stack, second UI framework
portal-shared-components	273 files, ~22,800 lines	MUI 7, Storybook, jQuery	Pulled in via a .d.ts alias

THE CORE CHANGE

Today, 36+ APIs attach tokens to five backends. Tomorrow, the frontend holds no tokens at all.

TARGET ARCHITECTURE

Dependencies point inward only



MODULE BOUNDARIES

Lint enforces the layers, not code review

Tag	May import	Holds
type:app	everything	main.tsx, router, store
type:feature	feature, state, ui, domain (types), util	Pages, containers
type:ui	ui, util	Presentational components
type:state	state, application, domain, infrastructure (hooks only), util	Slices, selectors, thunks
type:application	application, domain, util	Use cases
type:infrastructure	infrastructure, domain, util	portalApi, endpoints, adapters
type:domain	domain, util	Anemics, value objects, ports

Plus scope:<context> and scope:shared — a context imports only itself and shared libs.

AUTHENTICATION

Keycloak moves behind the API

Contract the API provides

Endpoint	Purpose
GET /auth/login	Code flow + PKCE, sets HttpOnly cookie
POST /auth/logout	Ends session and Keycloak login
GET /me	User, permissions, CSRF token or 401
GET /config	Branding, logo, feature flags
/api/**	All business calls, proxied backends

Frontend flow

1. Start: GET /me with credentials
2. 401: redirect to /auth/login
3. 200: session slice, menus filter by permissions
4. Any later 401: same central redirect
5. Refresh happens invisibly in the API

DATA FLOW

RTK Query lives in infrastructure

One createApi per app

portalApi with fetchBaseQuery replaces the 36+ legacy APIs. Contexts add endpoints via injectEndpoints.

Mapping at the edge

transformResponse turns DTOs into anemics. No DTO leaves infrastructure.

Use cases where there is logic

A port adapter calls endpoint.initiate(), so use cases and hooks share one cache.

```
export const marketplaceApi =
  portalApi.injectEndpoints({
    endpoints: (build) ⇒ ({
      activeApps: build.query<
        Array<AppAnemic>, void>({
        query: () ⇒ 'apps/active',
        transformResponse: (dtos) ⇒
          dtos.map(AppMapper.toAnemic),
        providesTags: ['App'],
      }),
    }),
  });
```

RUNTIME CONFIG

Build once, run in every environment

1. One Docker image, no API URL baked in
2. Kubernetes ConfigMap mounts /config.json into nginx
3. loadEnv() runs once before createRoot
4. Local dev falls back to import.meta.env

No window globals: code only reads ENV.apiBaseUrl.

```
export async function loadEnv(): Promise<void> {
  if (import.meta.env.DEV) {
    setEnv({ apiBaseUrl: import.meta.env.VITE_API_BASE_URL });
    return;
  }
  const response = await fetch('/config.json', {
    cache: 'no-store',
  });
  if (!response.ok) {
    throw new Error('config.json not available');
  }
  setEnv(await response.json());
}
```

TOOLING

Nx 23, TypeScript 6, ESLint over oxlint

Package	Version	Why
nx + @nx/*	23.3.0	Generators, boundaries
typescript	6.0.3	typescript-eslint needs < 6.1
eslint	9.39	React plugins not on 10 yet
react / react-router	19.2 / 8.5	Router 8 needs React 19.2.7+
@reduxjs/toolkit	2.13.0	Unchanged
openapi-typescript	7.13.0	DTOs from the OpenAPI spec
npm	—	Widely known for the hand-over

Rules that guard the architecture

- enforce-module-boundaries with type and scope tags
- @mui/* only in shared/ui-mui
- No React or Redux in domain and application
- curly: all, max-classes-per-file: 1

ROADMAP

Eight phases, each with a checkable done

PHASE 0

API contract

OpenAPI for auth and session, MSW mock

PHASE 1

Nx workspace

Empty app builds, lint fails on a layer breach

PHASE 2

Shared foundation

Layout, theme, i18n, http, config from libs

PHASE 3

Identity context

Login and logout via BFF, no keycloak-js

PHASE 4

Pilot: Marketplace

All layers end to end, 90% domain coverage

PHASE 5

Other contexts

Four waves, legacy APIs retired per wave

PHASE 6

Registration app

Same libs, same BFF session

PHASE 7

Drop shared lib

portal-shared-components and jQuery gone

RISKS

The biggest risk is the API timeline

Risk	Impact	Mitigation
API rewrite slips	No real endpoints to test	OpenAPI first, MSW mock, contract tests
Five layers feel like boilerplate	Team bypasses the rules	Context generator, pilot as template
Features import infra hooks	Layer boundary erodes	Tag rule, hooks only via state
Logic drifts into components	Business rules scattered	Logic only in application, tested
Permissions differ from old roles	Wrong menus visible	Mapping table, E2E per role
Shared components linger	jQuery and MUI 7 in bundle	Replace while migrating

DECISIONS

Settled before we write code

npm
Widely known; others continue after the hackathon

New workspace
portal-nx, scope @portal

Separate API domain
CORS with credentials, same main domain

Generated DTOs
From OpenAPI; handwritten only as fallback

React Router 8.5
Requires React 19.2.7 or newer

Feature lib per context
Created by the context generator

NEXT STEPS

The workspace exists, the app comes next

Done

- portal-nx created, scope @portal
- Nx 23.3.0, TypeScript 6.0.3, npm
- @nx/react and @nx/eslint added

Next

1. Generate apps/portal with Vite, Vitest, Playwright
2. ESLint tags and boundary matrix
3. Context generator, then phase 2
4. Agree the BFF contract with the API team